
MCCE: A Framework for Multi-LLM Collaborative Search in Discrete Spaces with Similarity-Filtered Preference Learning

Nian Ran^{*12} Zhongzheng Li^{*345} Yue Wang⁵ Qingsong Ran⁵⁶ Xiaoyuan Zhang⁵ Shikun Feng⁵
Richard Allmendinger¹ Xiaoguang Zhao³

Abstract

Multi-objective discrete optimization problems, such as molecular design, pose significant challenges due to their vast and unstructured combinatorial spaces. Traditional evolutionary algorithms often get trapped in local optima, while expert knowledge can provide crucial guidance for accelerating convergence. Large language models (LLMs) offer powerful priors and reasoning ability, making them natural optimizers when expert knowledge matters. However, closed-source LLMs, though strong in exploration, cannot update their parameters and thus cannot internalize experience. Conversely, smaller open models can be continually fine-tuned but lack broad knowledge and reasoning strength. We introduce Multi-LLM Collaborative Co-evolution (MCCE), a hybrid framework that unites a frozen closed-source LLM with a lightweight trainable model. The system maintains a trajectory memory of past search processes; the small model is progressively refined via reinforcement learning, with the two models jointly supporting and complementing each other in global exploration. Unlike model distillation, this process enhances the capabilities of both models through mutual inspiration. Experiments on multi-objective drug design benchmarks show that MCCE achieves state-of-the-art Pareto front quality and consistently outperforms baselines. These results highlight a new paradigm for enabling continual evolution in hybrid LLM systems, combining knowledge-

driven exploration with experience-driven learning. The code of MCCE is available on <https://github.com/lzz-z/MCCE>

1. Introduction

Discrete optimization and multi-objective optimization problems are pervasive in real-world applications, ranging from logistics and scheduling to scientific discovery and molecular design (Sun et al., 2025). These problems are notoriously difficult due to their vast, high-dimensional, and unstructured search spaces. Traditional evolutionary algorithms, while widely adopted, often suffer from two critical limitations: (i) they are prone to premature convergence, getting trapped in local optima, and (ii) they struggle to maintain both diversity and quality in the candidate population. These limitations highlight the need for more adaptive, intelligent optimization frameworks.

The rise of Large Language Models (LLMs) opens a promising direction. With their strong reasoning ability and broad prior knowledge, LLMs can act as powerful operators for generating and refining candidate solutions (Zhao et al., 2025). However, their application in iterative optimization remains constrained. First, a single LLM tends to converge to its own distribution, reducing solution diversity across generations (Li et al., 2025; Luo et al., 2025; Gao et al., 2025b). Second, although retrieval-augmented generation (RAG) enables the injection of external knowledge through contextual retrieval, it is inherently limited by the size of the context window and lacks the ability to update model parameters. As a result, such systems cannot genuinely accumulate knowledge or learn from past experiences. These challenges highlight that effective optimization requires not only problem-solving capacity but also mechanisms for internalizing feedback and continuously evolving. To this end, we argue that parameter training is indispensable. Unlike static prompting or RAG, parameter updates enable a model to accumulate experience in a much deeper and more persistent way. Yet, this poses a dilemma: closed-source LLMs excel in reasoning and general knowledge but cannot be fine-tuned, whereas small open-source models are trainable but

^{*}Equal contribution ¹University of Manchester ²UKRI AI Centre for Doctoral Training in Decision Making for Complex Systems ³Institute of Automation, Chinese Academy of Sciences ⁴School of Artificial Intelligence, University of Chinese Academy of Sciences ⁵Zhongguancun Academy ⁶Tongji University. Correspondence to: Yue Wang <yuewang@bza.edu.cn>, Xiaoyuan Zhang <zhangxiaoyuan@bza.edu.cn>.

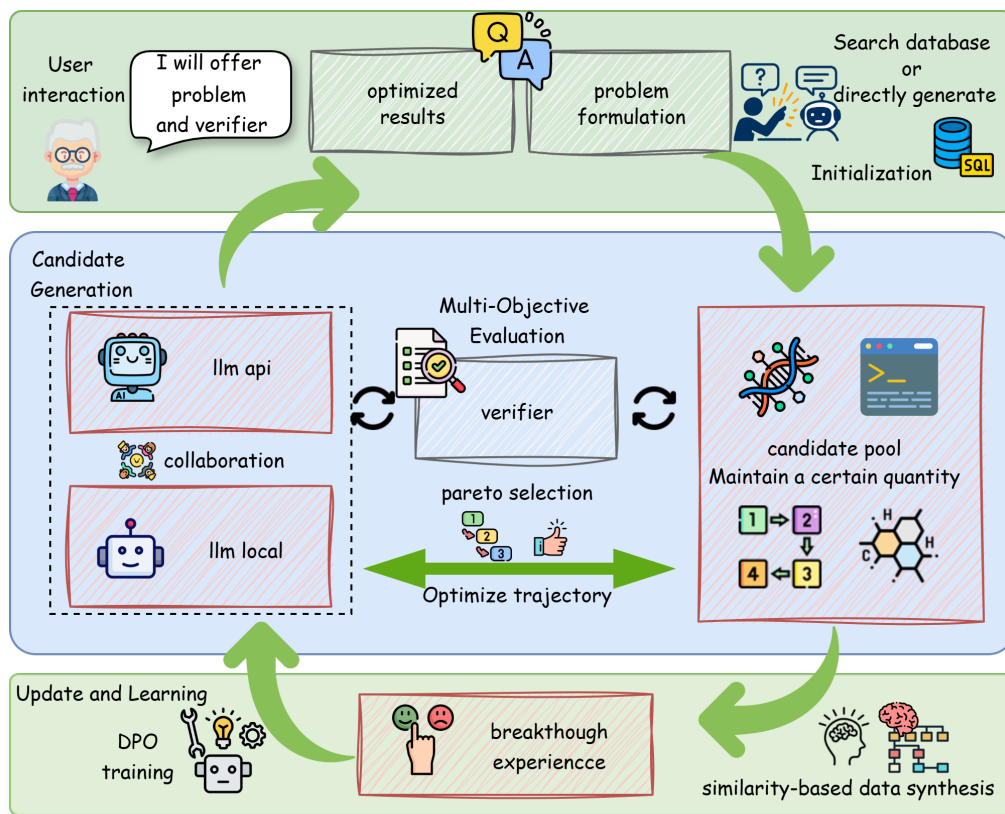


Figure 1. Overview of the proposed MCCE framework. The system starts from **user interaction** and **population initialization** based on the problem definition and evaluation criteria. During **candidate generation**, a frozen API-based LLM and a trainable local LLM collaboratively propose new molecules. These candidates are assessed by the **multi-objective evaluation** module, which applies Pareto-based selection to maintain a balanced population, while breakthrough solutions are stored as experience. In the **update and learning** stage, similarity-based data synthesis constructs preference pairs from historical trajectories, and the local model is refined via DPO training. This process forms a self-improving feedback loop in which global exploration (API LLM) and local adaptation (trainable LLM) co-evolve toward progressively optimized solutions.

lack the broad capabilities of larger models. Relying solely on either side leads to inherent inefficiency and bottlenecks.

This motivates our proposed Multi-LLM Collaborative Co-evolution (MCCE) framework, a system where a frozen, closed-source LLM and a lightweight, trainable local model co-evolve through iterative collaboration. In each generation, the two models alternate as evolutionary operators: the closed-source LLM drives global exploration, while the local model learns from accumulated experiences to perform more targeted searches. Crucially, we design a feedback loop where the local model is periodically refined using breakthrough search trajectories, ensuring that knowledge is continually internalized and reused. Unlike traditional distillation, our framework establishes mutual inspiration between models—large models provide global guidance, while small models adaptively extend the search frontier through learning. Recent work such as ExLLM (Ran et al., 2025) has demonstrated the promise of using LLMs as evolutionary operators for multi-objective molecular design, combining in-context learning with prompt engineering to

achieve strong results. However, these approaches still rely on a single frozen LLM, which limits their ability to accumulate experience through parameter updates and often leads to reduced diversity and premature convergence. In contrast, our MCCE framework explicitly addresses this gap by coupling a powerful but fixed closed-source LLM with a lightweight trainable model. This collaborative co-evolution not only preserves the broad reasoning and exploration capacity of large models, but also equips the system with a mechanism for continual learning and adaptation. By enabling mutual inspiration between heterogeneous models, the closed-source LLM continuously proposes better molecules under improved prompts, while simultaneously generating high-quality data for training. The local LLM learns from these experiences, improves its ability to generate better candidates, and in turn provides stronger candidates back to the closed-sourced LLM as context. Under this co-evolutionary collaboration, the framework achieves stronger state-of-the-art performance under a fixed budget, while both models consistently outperform their standalone

counterparts.

The main contributions of this paper are:

1. **A collaborative co-evolution framework (MCCE).** We integrate closed-source LLMs with lightweight, trainable local models, combining the exploration capacity of large models with the adaptability of smaller models. This hybrid design is broadly applicable to discrete, multi-objective optimization tasks beyond drug discovery.
2. **An experience-driven learning paradigm.** We leverage breakthrough evolutionary trajectories as valuable experience, guiding the local model to identify promising search directions. This cooperative mechanism allows the global and local models to co-evolve, reinforcing each other’s strengths over time.
3. **Demonstrated practical efficacy and extensibility.** Our framework achieves state-of-the-art performance in multi-objective drug design, highlighting its potential for real-world impact. Moreover, the paradigm is extensible to a wider range of scientific and engineering domains where structured optimization is critical.

2. Methodology

We propose Multi-LLM Collaborative Co-evolution (MCCE), a unified and general-purpose optimization framework for complex discrete problems, demonstrated here in molecular design. As shown in Figure 1, the system operates through an iterative collaboration between two distinct LLMs: a powerful but frozen model and a lightweight, trainable local model. The frozen LLM provides robust global exploration, while the local model continuously refines its policy by learning from successful search trajectories, forming a self-improving feedback loop. To validate MCCE, we adopt a challenging five-objective molecular optimization task, jointly targeting *QED*, *synthetic accessibility (SAscore)*, *DRD2 binding*, *GSK3 β binding*, and *JNK3 binding*. This setting builds on recent benchmarks such as ExLLM (Ran et al., 2025) and MoLLEO (Wang et al., 2024), which emphasize that realistic drug discovery requires balancing multiple properties. While MoLLEO showed the benefit of LLM-based evolutionary operators, its evaluation was restricted to three objectives. By extending to five objectives, we align with prior work while pushing toward more realistic, high-dimensional challenges, providing a rigorous test of MCCE’s adaptability.

2.1. Overall Framework

The proposed MCCE framework operates in an iterative evolutionary loop, where large language models (LLMs)

act as adaptive genetic operators. The overall process can be divided into four key stages: initialization, generation, evaluation, and update with learning.

Stage 1: Initialization. Let $\mathcal{P}_t$ denote the population pool at generation t , consisting of candidate molecules. The process begins with an initial population $\mathcal{P}_0$, which can be sampled either from an external database or generated by a pretrained LLM:

$$\mathcal{P}_0 = \{c_1, c_2, \dots, c_M\}, \quad c_i \sim \pi_{\text{init}}(\cdot), \quad (1)$$

where π_{init} represents the initialization distribution.

Stage 2: Candidate Generation. At each generation t , two parents $p_1, p_2 \in \mathcal{P}_t$ are selected according to a selection strategy (e.g., tournament or fitness-proportional selection). Given the pair (p_1, p_2) and a task-specific prompt function $\text{prompt}(p_1, p_2)$, the LLM-based operator produces two new candidates:

$$(c_1, c_2) \sim \pi_{\text{LLM}}(\cdot \mid \text{prompt}(p_1, p_2)). \quad (2)$$

Since each invocation of the operator generates exactly two candidates, constructing a full population of size M requires

$$\frac{M}{2} \text{ generations of prompts.} \quad (3)$$

This process is repeated with different parent pairs until the entire offspring set is produced. The operator π_{LLM} alternates between a frozen API model and a locally trainable model, thereby balancing *global exploration* (via frozen LLM) and *local adaptation* (via trainable LLM).

Stage 3: Multi-Objective Evaluation. Each generated candidate c is evaluated using a multi-objective scoring function:

$$\mathbf{s}(c) = [s_1(c), s_2(c), \dots, s_K(c)], \quad (4)$$

where $s_k(c)$ is the score under the k -th objective (e.g., drug-likeness, synthesizability, or binding affinity). All scores are normalized to a common scale:

$$\hat{s}_k(c) = \frac{s_k(c) - \mu_k}{\sigma_k}, \quad (5)$$

where μ_k and σ_k are the mean and standard deviation of scores in the current population.

Stage 4: Update and Learning. The next-generation population $\mathcal{P}_{t+1}$ is formed by applying Pareto front selection, which preserves non-dominated solutions while maintaining diversity. Meanwhile, after every N generated candidates, successful trajectories

$$(\text{prompt}(p_1, p_2) \rightarrow (c_1, c_2) \rightarrow \mathbf{s}(c_1), \mathbf{s}(c_2))$$

Table 1. Overall performance comparison on multi-objective optimization tasks. This table compares internal training mechanisms and representative baseline methods. Results are reported as mean $\pm$ std over 5 runs. Best results are shown in **bold**, and second-best results are underlined.

Model	Top1 F	Top10 F	AUC-Top10	HV	Diversity	Uniqueness	Validity
<i>Internal Mechanisms & Training Paradigms</i>							
qwen2.5-7b-instruct	4.07 $\pm$ 0.04	4.05 $\pm$ 0.04	3.91 $\pm$ 0.03	0.516 $\pm$ 0.102	0.543 $\pm$ 0.047	0.576 $\pm$ 0.018	0.838 $\pm$ 0.025
gpt-4o-2024-05-13	4.16 $\pm$ 0.15	4.14 $\pm$ 0.12	3.99 $\pm$ 0.08	0.661 $\pm$ 0.214	0.497 $\pm$ 0.035	0.702 $\pm$ 0.056	0.902 $\pm$ 0.022
collaboration	4.19 $\pm$ 0.15	4.13 $\pm$ 0.12	3.96 $\pm$ 0.06	0.695 $\pm$ 0.189	0.524 $\pm$ 0.048	0.750 $\pm$ 0.041	0.838 $\pm$ 0.024
rl.coevolve	4.19 $\pm$ 0.17	4.16 $\pm$ 0.15	3.99 $\pm$ 0.09	0.709 $\pm$ 0.219	0.509 $\pm$ 0.059	0.683 $\pm$ 0.045	0.893 $\pm$ 0.021
sft.coevolve	4.24 $\pm$ 0.25	4.20 $\pm$ 0.22	3.99 $\pm$ 0.11	0.709 $\pm$ 0.288	0.478 $\pm$ 0.070	0.571 $\pm$ 0.047	0.905 $\pm$ 0.020
MCCE (dpo.coevolve)	4.35 $\pm$ 0.17	4.28 $\pm$ 0.15	<u>4.02 $\pm$ 0.09</u>	<u>0.847 $\pm$ 0.138</u>	0.484 $\pm$ 0.063	0.660 $\pm$ 0.018	<u>0.820 $\pm$ 0.022</u>
- dpo.coevolve:local	4.27 $\pm$ 0.16	4.22 $\pm$ 0.14	4.01 $\pm$ 0.03	0.826 $\pm$ 0.126	0.555 $\pm$ 0.055	0.633 $\pm$ 0.025	0.759 $\pm$ 0.030
- dpo.coevolve:api	4.35 $\pm$ 0.17	4.28 $\pm$ 0.14	4.03 $\pm$ 0.07	0.855 $\pm$ 0.135	<u>0.505 $\pm$ 0.062</u>	0.784 $\pm$ 0.016	0.907 $\pm$ 0.016
<i>Comparison with Baselines</i>							
GB-GA(Jensen, 2019)	4.02 $\pm$ 0.10	3.98 $\pm$ 0.10	3.86 $\pm$ 0.05	0.643 $\pm$ 0.268	0.623 $\pm$ 0.047	0.821 $\pm$ 0.032	1.000 $\pm$ 0.000
REINVENT(Olivecrona et al., 2017)	4.23 $\pm$ 0.20	4.14 $\pm$ 0.22	3.93 $\pm$ 0.13	0.742 $\pm$ 0.259	0.640 $\pm$ 0.111	0.690 $\pm$ 0.132	0.979 $\pm$ 0.002
MoLLEO(Wang et al., 2024)	4.19 $\pm$ 0.08	4.08 $\pm$ 0.02	3.95 $\pm$ 0.02	0.860 $\pm$ 0.088	0.670 $\pm$ 0.015	0.575 $\pm$ 0.075	0.938 $\pm$ 0.007
GFlowNet(Kim et al., 2024)	4.24 $\pm$ 0.25	4.20 $\pm$ 0.21	4.08 $\pm$ 0.15	0.871 $\pm$ 0.288	0.633 $\pm$ 0.066	0.349 $\pm$ 0.004	0.998 $\pm$ 0.000
DyMol(Shin et al., 2024)	4.23 $\pm$ 0.17	4.16 $\pm$ 0.13	4.00 $\pm$ 0.05	0.868 $\pm$ 0.146	0.581 $\pm$ 0.069	0.986 $\pm$ 0.005	1.000 $\pm$ 0.000
MCCE (Ours)	4.35 $\pm$ 0.17	4.28 $\pm$ 0.15	<u>4.02 $\pm$ 0.09</u>	0.847 $\pm$ 0.138	0.484 $\pm$ 0.063	0.660 $\pm$ 0.018	0.820 $\pm$ 0.022

are stored as experience $\mathcal{D}$. This dataset is then used to refine the trainable LLM. Formally, the model parameters are updated as

$$\pi_{\text{LLM}} \leftarrow \text{Update}(\pi_{\text{LLM}}, \mathcal{D}), \quad (6)$$

where $\text{Update}(\cdot)$ denotes an abstract learning procedure based on the accumulated experience. This establishes a closed-loop cycle of *generation-evaluation-learning-evolution*.

2.2. Which Training Paradigm Best Supports Experience-Driven Learning?

A central question in our framework is how to effectively refine the local model’s policy through accumulated experience. To this end, we systematically explored several candidate training paradigms and evaluated their suitability for stabilizing learning while preserving the model’s exploratory capacity. Our findings reveal critical limitations in conventional approaches:

Supervised Fine-Tuning (SFT). We first adopted SFT by treating “breakthrough” generations as positive training samples. Concretely, if a generated molecule achieved a score higher than all of its parents, the corresponding trajectory was labeled as effective data. However, this approach led to catastrophic forgetting: after training, the uniqueness of generated molecules dropped substantially. This indicates that the local model tended to memorize successful chemical formulas rather than internalize a generalizable exploration strategy, thereby losing its ability to propose genuinely novel solutions.

Reinforcement Learning (RL). Next, we experimented with reinforcement learning using the scoring function as the reward signal. In practice, this training proved highly unstable. Strong negative rewards for low-scoring molecules

caused the model to collapse, as it struggled to infer the underlying reasons for the penalties and consequently lost its ability to generate valid candidates. The mapping between molecular structures and their scores is inherently unpredictable for an LLM, making explicit quantitative rewards unsuitable for stable RL training in this context.

Direct Preference Optimization (DPO). To overcome these issues, we adopted a DPO-based approach, which provides a more stable and sample-efficient training signal without requiring an explicit reward model. Initially, we constructed training pairs by contrasting high-scoring versus low-scoring molecules under the same prompt. However, we observed unstable loss oscillations: since identical prompts were associated with conflicting responses, the model often became confused. To address this, we developed a *similarity-based data synthesis* method, which ensures that preference pairs are constructed from structurally comparable molecules. This adjustment significantly improved both training stability and data efficiency. The details of this method are elaborated in Section 4.3.

2.3. Similarity-Based Data Synthesis

Our DPO training requires triplets of the form (q, τ^+, τ^-) where q is a query (prompt), τ^+ is a preferred (chosen) trajectory and τ^- is a rejected trajectory. To construct such triplets stably and to mitigate distributional shift between the frozen API model and the local trainable model, we propose a similarity-based data synthesis pipeline. The pipeline proceeds in three phases: (1) collect candidate pool and compute similarity statistics, (2) filter and stratify candidates by score and similarity, (3) assemble DPO triplets with fallback rules.

Notation. Let $\mathcal{H} = \{q_1, q_2, \dots, q_{|H|}\}$ be the historical prompts (ordered by time). For each prompt q_j we have

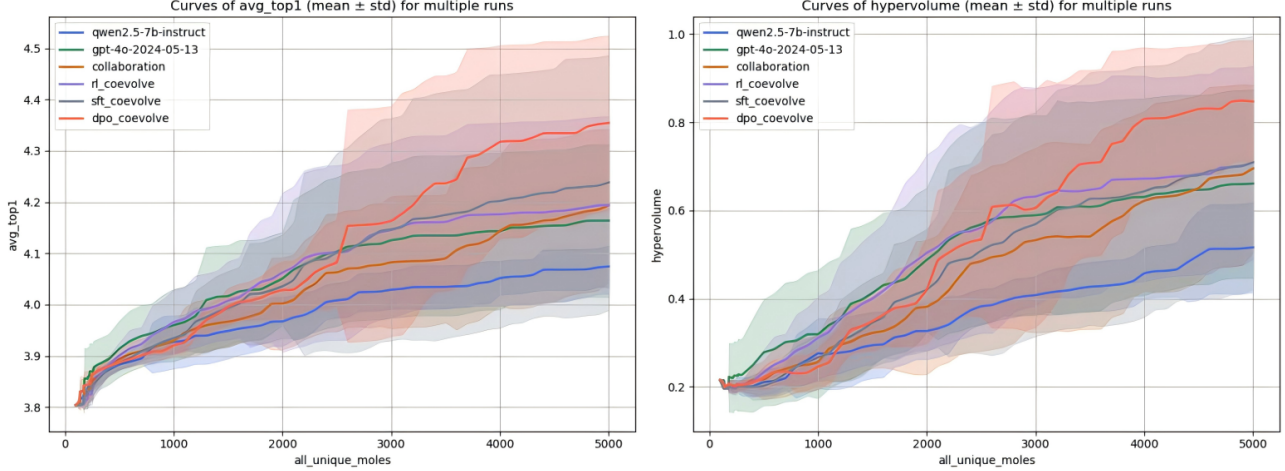


Figure 2. Overall performance comparison across different baselines.(Left) The curve of **avg_top1** (mean $\pm$ std) shows that our DPO-enhanced co-evolutionary framework consistently outperforms all baselines, steadily increasing the average quality of the top-ranked molecule throughout the optimization process.(Right) The curve of **hypervolume** (mean $\pm$ std) further highlights the superiority of our approach: MCCE with DPO training achieves the largest Pareto front coverage, demonstrating both improved solution quality and diversity.In both metrics, our method significantly surpasses single-model baselines (e.g., Qwen2.5-7B-Instruct, GPT-4o-2024-05-13) as well as alternative co-evolution variants (SFT and RL), achieving state-of-the-art performance.

a set of generated candidates $\mathcal{C}_j = \{c_{j,1}, c_{j,2}, \dots\}$, produced by either the frozen LLM or the local model during the recent evolution window. Let $s(c)$ denote the (multi-objective) score of candidate c (we use a scalarized score or a ranking for stratification). Define a molecular similarity function $\text{sim}(c, q) \in [0, 1]$, computed by a fingerprint-based metric (e.g., Tanimoto on Morgan fingerprints) or any task-appropriate similarity $\phi(\cdot, \cdot)$.

Phase 1 — similarity statistics. Collect the similarity values across the considered history and models:

$$\mathcal{S} = \{\text{sim}(c, q) : q \in \mathcal{H}, c \in \mathcal{C}_q\}.$$

Compute the empirical mean and standard deviation:

$$\mu = \frac{1}{|\mathcal{S}|} \sum_{x \in \mathcal{S}} x, \quad \sigma = \sqrt{\frac{1}{|\mathcal{S}|} \sum_{x \in \mathcal{S}} (x - \mu)^2}. \quad (7)$$

We will use (μ, σ) as global similarity statistics to reduce distributional mismatch between models (both models' outputs contribute to $\mathcal{S}$).

Define a global similarity filter:

$$\mathcal{F} = \{c \mid \mu - \sigma \leq \text{sim}(c, q) \leq \mu + \sigma\}. \quad (8)$$

Only candidates in $\mathcal{F}$ are considered for DPO pair construction (this ensures candidates are within one standard deviation of the empirical similarity distribution).

Phase 2 — score stratification and similarity windows. Let α denote the quantile threshold used to form top/bottom

pools (we use $\alpha = 0.3$ by default). Let $\mathcal{C}_{\text{all}} = \bigcup_q \mathcal{C}_q$ and sort $\mathcal{C}_{\text{all}}$ by score $s(\cdot)$. Define

$$\begin{aligned} \mathcal{T}_{\text{high}} &= \{\text{top } \alpha \text{ fraction of } \mathcal{C}_{\text{all}}\}, \\ \mathcal{T}_{\text{low}} &= \{\text{bottom } \alpha \text{ fraction of } \mathcal{C}_{\text{all}}\}. \end{aligned} \quad (9)$$

We further define nested similarity intervals (from strict to relaxed):

$$\begin{aligned} I_1 &= [\mu + \frac{2}{3}\sigma, \mu + \sigma], \\ I_2 &= [\mu + \frac{1}{3}\sigma, \mu + \sigma], \\ I_3 &= [\mu, \mu + \sigma]. \end{aligned} \quad (10)$$

These intervals prioritize chosen candidates that are both high-scoring and reasonably similar to the prompt (thus reducing contradictory prompt-response pairs that destabilize training).

Phase 3 — per-prompt pair construction with fallback rules. To construct stable DPO training triplets, we design a per-prompt pair construction algorithm that selects a preferred (τ^+) and a rejected (τ^-) candidate for each prompt q . As outlined in Algorithm 1, the procedure first filters candidates by similarity, then attempts to select τ^+ from the high-score pool and τ^- from the low-score pool using progressively relaxed similarity intervals ($I_1 \rightarrow I_2 \rightarrow I_3$), and finally falls back to broader score ranges (Top/Bottom-50%) if no candidates are available. Each valid pair yields a triplet (q, τ^+, τ^-) used for DPO training.

For clarity, we provide in the main text a simplified version of the algorithm, while a fully detailed pseudocode with all implementation nuances and fallback rules is presented in

Table 2. Ablation studies of MCCE, analyzing the impact of similarity strategy, API/local call split, and update frequency. Results are reported as mean $\pm$ std over 5 runs. Best results are shown in **bold**, and second-best results are underlined.

Setting	Top1 F	Top10 F	AUC-Top10	HV	Diversity	Uniqueness	Validity
<i>Ablation: Similarity Strategy</i>							
dpo_coevolve ($\alpha = 0.30$)	4.35 $\pm$ 0.17	4.28 $\pm$ 0.15	4.02 $\pm$ 0.09	0.847 $\pm$ 0.138	0.484 $\pm$ 0.063	0.660 $\pm$ 0.018	0.820 $\pm$ 0.022
dpo_coevolve ($\alpha = 0.30$, only I_3)	4.32 $\pm$ 0.14	4.21 $\pm$ 0.10	3.99 $\pm$ 0.08	0.848 $\pm$ 0.086	0.532 $\pm$ 0.059	0.694 $\pm$ 0.058	<u>0.829 $\pm$ 0.022</u>
dpo_coevolve ($\alpha = 0.40$)	4.36 $\pm$ 0.12	4.27 $\pm$ 0.07	3.99 $\pm$ 0.05	0.843 $\pm$ 0.091	0.462 $\pm$ 0.009	0.664 $\pm$ 0.072	0.816 $\pm$ 0.025
dpo_coevolve ($\alpha = 0.20$)	4.33 $\pm$ 0.11	4.26 $\pm$ 0.06	4.02 $\pm$ 0.06	0.828 $\pm$ 0.091	0.483 $\pm$ 0.030	0.651 $\pm$ 0.052	0.836 $\pm$ 0.047
dpo_coevolve (embedding)	4.29 $\pm$ 0.12	4.22 $\pm$ 0.07	3.98 $\pm$ 0.05	<u>0.847 $\pm$ 0.104</u>	<u>0.515 $\pm$ 0.060</u>	<u>0.688 $\pm$ 0.038</u>	0.825 $\pm$ 0.023
<i>Ablation: Call Split (API/Local)</i>							
50/50	4.31 $\pm$ 0.12	4.22 $\pm$ 0.08	3.99 $\pm$ 0.05	0.838 $\pm$ 0.120	0.453 $\pm$ 0.061	0.641 $\pm$ 0.082	0.816 $\pm$ 0.038
50/32	4.35 $\pm$ 0.17	4.28 $\pm$ 0.15	4.02 $\pm$ 0.09	0.847 $\pm$ 0.138	0.484 $\pm$ 0.063	0.660 $\pm$ 0.018	0.820 $\pm$ 0.022
50/16	4.25 $\pm$ 0.20	4.19 $\pm$ 0.15	3.96 $\pm$ 0.06	0.734 $\pm$ 0.249	<u>0.462 $\pm$ 0.015</u>	<u>0.656 $\pm$ 0.111</u>	0.809 $\pm$ 0.056
<i>Ablation: Update Frequency</i>							
500 candidates	4.28 $\pm$ 0.17	4.21 $\pm$ 0.12	3.98 $\pm$ 0.06	0.796 $\pm$ 0.205	0.493 $\pm$ 0.054	0.658 $\pm$ 0.099	0.848 $\pm$ 0.048
200 candidates	4.32 $\pm$ 0.17	4.27 $\pm$ 0.16	4.02 $\pm$ 0.11	0.856 $\pm$ 0.160	0.500 $\pm$ 0.055	0.626 $\pm$ 0.033	0.823 $\pm$ 0.019
1_round	4.35 $\pm$ 0.17	4.28 $\pm$ 0.15	<u>4.02 $\pm$ 0.09</u>	<u>0.847 $\pm$ 0.138</u>	<u>0.484 $\pm$ 0.063</u>	0.660 $\pm$ 0.018	0.820 $\pm$ 0.022
<i>Ablation: 2API</i>							
GPT-4o + Gemini	4.11 $\pm$ 0.10	4.06 $\pm$ 0.08	3.96 $\pm$ 0.06	0.492 $\pm$ 0.083	0.564 $\pm$ 0.071	0.515 $\pm$ 0.034	0.946 $\pm$ 0.007

Appendix, ensuring reproducibility and transparency of our method.

Dataset and hyperparameters. Let L be the number of recent prompts used and r the number of pairs per prompt (default $r = 1$). The resulting DPO dataset size is at most $D \leq L \cdot r$. The key hyperparameters are α (score quantile, default 0.3), the similarity relaxation windows I_1, I_2, I_3 , and the global similarity acceptance band $\mu \pm \sigma$. These are chosen to (i) favor high-quality examples, (ii) ensure chosen/rejected pairs are structurally comparable, and (iii) avoid pairing identical prompt with widely varying responses that confuse the learner.

Why this reduces distribution shift. By (a) computing μ, σ from the union of both models’ outputs, (b) enforcing the global similarity filter $\mathcal{F}$, and (c) selecting chosen/rejected candidates from narrow similarity windows, we ensure that the training pairs are consistent with the local model’s typical output distribution. This reduces the likelihood that the local model is asked to map a single prompt to mutually contradictory responses and therefore stabilizes DPO optimization.

3. Experiments

3.1. Experimental Setup

We evaluate MCCE in the domain of multi-objective drug design, a highly challenging problem that requires navigating an enormous chemical space to identify molecules balancing multiple, often conflicting, properties. For the frozen, closed-source LLMs, we leveraged the GPT-4o-2024-05-13 and Gemini-2.5-flash-nothinking models through their APIs, while the local trainable component was instantiated with Qwen2.5-7B-Instruct. The initial population of candidate molecules was constructed by randomly sampling

100 molecules from the ZINC dataset, ensuring sufficient diversity at the start of evolution. The generated molecules were assessed against five standard drug-likeness objectives, and the final optimization outcome was measured using the Hypervolume Indicator (HV), a widely adopted metric in multi-objective optimization that jointly reflects solution quality and diversity. For training paradigms, we implemented SFT and RL baselines using the `verl` library, while our DPO method was implemented with the `trl` library to ensure stable preference-based optimization.

3.2. Main Results

3.2.1. OVERALL PERFORMANCE

Table 1 provides a comprehensive evaluation across three critical dimensions. First, in terms of Internal Mechanisms, our DPO-driven approach significantly outperforms both single-model baselines and alternative training paradigms (SFT and RL). It achieves the highest Top-1 Fitness by effectively mitigating catastrophic forgetting and training instability. Second, compared against SOTA Baselines such as GFlowNet and DyMol, MCCE demonstrates superior optimization capability. It consistently discovers molecules with higher fitness while maintaining competitive diversity and validity scores. Finally, the Ablation Studies validate the optimality of our design choices, confirming that the proposed similarity-based data synthesis ($\alpha = 0.30$), asymmetric collaboration split (50/32), and frequent model updates are essential factors for maximizing system performance. Figure 2 and Table 1 present a clear comparison of the collaborative system’s performance with and without parameter training, unequivocally demonstrating that the continuous learning mechanism is crucial for long-term optimization gains.

3.2.2. THE CO-EVOLUTIONARY CURVE AND OUTPUT DISTRIBUTION ANALYSIS

To highlight the effectiveness of our collaborative design, we present two complementary visualizations in Figure 3.

(Left) The co-evolutionary curve. This curve captures the dynamics of how the frozen large LLM and the fine-tuned local model collaborate throughout the optimization process. The large LLM consistently provides broad global exploration, generating diverse candidates guided by its rich prior knowledge. In parallel, the local model—refined through iterative learning from breakthrough trajectories—adapts to the search space and performs targeted exploitation. The alternating interplay between these two roles prevents premature convergence, increases diversity, and steadily drives the optimization toward superior regions of the search space. The curve clearly illustrates that their collaboration outperforms the trajectory of either model alone.

(Right) Output distribution analysis. To further examine the learning effect, we analyze the quality distribution of molecules generated by three models: the frozen LLM, the initial (untrained) local model, and the fine-tuned local model. Using a no-parent prompt, we sample 1,000 molecules from each model. The histogram shows that the trained local model produces a distribution shifted significantly toward higher scores, surpassing both the frozen LLM and the untrained local baseline. This confirms that the fine-tuning procedure successfully internalizes experience, allowing the local model to approximate the distribution of high-quality molecules. Combined with the steadily decreasing training loss, this analysis demonstrates that our framework not only generates strong solutions but also achieves continual improvement through experience-driven learning.

Algorithm 1 Simplified Per-Prompt DPO Pair Construction

```

1: Input: Recent prompts  $\mathcal{H}$ , candidate sets  $\{\mathcal{C}_q\}$ 
2: Output: Triplets  $(q, \tau^+, \tau^-)$ 
3: Select the most recent  $L$  prompts from  $\mathcal{H}$ 
4: for each prompt  $q$  do
5:   Filter candidates to obtain  $\mathcal{C}_q^{\mathcal{F}}$ 
6:   Sample  $\tau^+$  from the high-score pool with similarity level
7:      $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow \text{top } 50\%$ 
8:   Sample  $\tau^-$  from the low-score pool with similarity level
9:      $I_1 \rightarrow I_2 \rightarrow I_3 \rightarrow \text{bottom } 50\%$ 
10:  Record triplet  $(q, \tau^+, \tau^-)$ 
11: end for

```

3.3. Orthogonal Validation with Docking Scores

We further evaluate the optimized molecules with AutoDock Vina docking scores as an orthogonal validation beyond the benchmark predictors. In our benchmark, the DRD2, JNK3, and GSK3 β objectives are computed using pretrained machine-learning predictors from TDC, whereas AutoDock Vina provides a structure-based docking score with a different scoring mechanism. Since SA and QED are rule-based objectives rather than target-binding predictors, we focus the docking validation on DRD2, JNK3, and GSK3 β .

As shown in Table 3, the final top-10 molecules achieve clear improvements on the benchmark objectives. SA and DRD2 decrease, QED and JNK3 increase substantially, and the overall fitness improves from 3.486 to 4.281. GSK3 β shows a slight increase, indicating a non-trivial trade-off in the five-objective optimization setting.

Table 4 reports the corresponding docking-based validation. Since lower AutoDock Vina scores indicate stronger binding affinity, the desired direction is target-dependent: JNK3 should become more negative, while DRD2 and GSK3 β should become less negative. The docking results are consistent with the main optimization trend. For JNK3, the benchmark score increases from 0.015 to 0.624, and the docking score also becomes more favorable for binding, changing from -7.356 to -8.585. For DRD2, the benchmark score decreases from 0.009 to 0.003, and the docking score becomes less negative, changing from -8.400 to -8.141. For GSK3 β , although the TDC proxy score slightly increases, the docking score becomes less negative, changing from -7.791 to -7.625, suggesting that stronger off-target binding is not observed under the orthogonal evaluator. Overall, these results provide additional evidence that MCCE improves molecular candidates through meaningful structural optimization rather than merely exploiting a single proxy evaluator.

3.4. Generalization to Combinatorial Optimization

To further validate the universality of the MCCE framework beyond molecular design, we extended our evaluation to three classic NP-hard problems: the Circle Packing problem, the Multi-Objective Traveling Salesman Problem (MOTSP), and the Multi-Objective Capacitated Vehicle Routing Problem (MOCVRP).

Circle Packing: Requires placing n non-overlapping circles in a unit square to maximize the common radius. We compare our results against long-standing community records.

MOTSP: Seeks a single Hamiltonian circuit that, starting and ending at a given depot, visits every city exactly once while simultaneously minimizing multiple conflicting objectives.

Table 3. Comparison between the initial population and the final top-10 molecules under the benchmark property scores.

Split	SA ↓	DRD2 ↓	QED ↑	GSK3β ↓	JNK3 ↑	Fitness ↑
Initial population	3.021 ± 0.792	0.009 ± 0.033	0.730 ± 0.143	0.026 ± 0.040	0.015 ± 0.022	3.486 ± 0.166
Final top-10	1.818 ± 0.162	0.003 ± 0.004	0.795 ± 0.012	0.045 ± 0.027	0.624 ± 0.034	4.281 ± 0.015

Table 4. AutoDock Vina docking scores for the initial population and the final top-10 molecules. More negative values indicate stronger predicted binding affinity.

Target	Initial docking	Final top-10 docking
DRD2	-8.400 ± 1.222	-8.141 ± 0.368
GSK3β	-7.791 ± 1.038	-7.625 ± 0.304
JNK3	-7.356 ± 1.034	-8.585 ± 0.358

Table 5. Hypervolume comparison on Combinatorial Optimization. MCCE achieves SOTA performance on MOCVRP and remains highly competitive on MOTSP against recent baselines.

Method	MOTSP ($n = 100$)	MOCVRP ($n = 100, m = 20$)
Pymoo	0.983488	0.955802
AIDE	1.020798	1.005552
FunSearch	1.023301	1.032126
ReEvo	1.028890	1.034541
AlphaEvolve	1.029279	1.031803
MCCE	1.025206	1.048843

MOCVRP: Designs a set of capacitated vehicle routes that originate from a common depot, serve all customer demands, and jointly minimize the total travel distance and the makespan.

Benchmark instances for the combinatorial tasks were generated following (Lin et al., 2022). We adopt Hyper-volume (HV) as the performance indicator. Baselines include the solver Pymoo (Blank & Deb, 2020) and recent search-based algorithms such as ReEvo, AlphaEvolve, AIDE (Jiang et al., 2025), and FunSearch. The results are shown in Table 5 and Table 6.

4. Related Work

4.1. Multi-Model Collaboration

Recent studies highlight the promise of collective intelligence in enhancing reasoning and problem-solving through multiple LLMs (Ye et al., 2025). For example, Misaki et al. (2025) propose an adaptive branching Monte Carlo Tree Search (MCTS) framework where multiple models cooperate to balance exploration and exploitation. Extending this collaborative paradigm to scientific discovery, Su et al. (2025) employ a multi-agent system to mimic human teamwork for generating and refining novel research ideas. By leveraging diverse model perspectives in the search pro-

Table 6. Comparison of MCCE against current community records for the Circle Packing problem. MCCE successfully discovers configurations surpassing the previous best-known results.

Size	Current Record	MCCE
$n = 26$	2.634+	2.635983
$n = 31$	2.889+	2.889970

cess, these approaches significantly improve efficiency and robustness compared to using a single LLM. Beyond inference scaling, other works explore multi-agent or ensemble strategies. Yang et al. (2025) demonstrate that integrating diverse reasoning pathways improves search-based reasoning, while Gao et al. (2025a) show the benefits of cross-model collaboration in structure-based drug design. Similarly, ensemble methods such as Huang et al. (2024) and Wang et al. (2023) propose novel ways to combine outputs or probability distributions across heterogeneous LLMs. While most aforementioned methods treat models as static entities, recent works have begun to incorporate training into the collaborative loop. For instance, Wu et al. (2025b) introduce reinforcement fine-tuning to transform models from passive responders into active collaborators. regarding heterogeneous collaboration, Lu et al. (2025) fine-tune small models to orchestrate fixed LLMs for cost-effective data labeling, and Xu et al. (2025) train small models to decompose queries to assist black-box LLMs in retrieval tasks. However, these approaches typically limit parameter updates to a single side of the collaboration or distinct functional modules. In contrast, our work emphasizes dynamic co-evolution, where both large and small models jointly learn and evolve on the same optimization task through shared experience.

4.2. Experience Learning

The ability of LLM-based agents to continuously learn from experience has been recognized as a critical step toward AGI (Zheng et al., 2025). Several approaches explore reinforcement learning (RL) as a means of improving reasoning. For example, ML-agent (Liu et al., 2025b) apply online RL for autonomous machine learning engineering, while CALM (Huang et al., 2025) and EvoTune (Surina et al., 2025) combine RL with evolutionary search to refine heuristics and algorithms. However, traditional RL often struggles with the capability boundary of base models. Works such as RL-PLUS (Dong et al., 2025) and LUFFY (Yan et al., 2025)

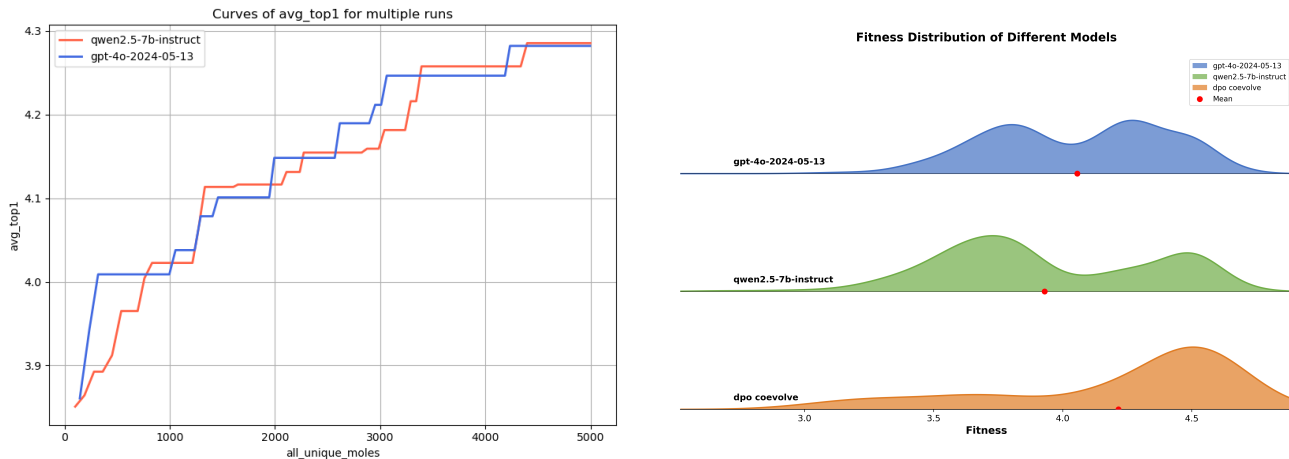


Figure 3. (Left) The co-evolutionary curve showing how the large LLM and local model complement each other to achieve superior trajectories. (Right) Output distribution analysis of molecules generated from the frozen LLM, the initial local model, and the fine-tuned local model.

address this by introducing hybrid-policy optimization or off-policy guidance. Complementary strategies, including ReLIFT (Ma et al., 2025) and TAPO (Wu et al., 2025a), integrate supervised fine-tuning or structured external guidance to capture knowledge beyond the reach of RL. These methods show that a single LLM can incrementally improve through experience, but they remain limited by the inherent ceiling of one model. Our approach differs by enabling multi-model collaborative experience learning, where small models benefit from learning while also enriching the exploration capacity of larger models, forming a co-evolutionary loop.

4.3. Evolutionary Algorithms

A rapidly growing body of work explores integrating LLMs with evolutionary algorithms for optimization and design. For example, FunSearch (Romera-Paredes et al., 2024), EoH (Liu et al., 2024) and MEoH (Yao et al., 2025) demonstrate that LLMs can serve as generators for heuristics or algorithms in combinatorial optimization problems. Reflective mechanisms further enhance search efficiency, as seen in REEVO (Ye et al., 2024) and ML-master (Liu et al., 2025a), where memory or reflection guides iterative exploration. Evolutionary methods have also been applied in specialized domains, including prompt evolution for jailbreak attacks (Liu et al., 2023) or over-refusal mitigation (Wu et al., 2025c). More recent works such as Alphaevolve (Novikov et al., 2025) and Dat et al. (2025) introduce evaluator feedback loops, but still treat LLMs as static generators within the search process. Overall, while these studies validate the synergy between LLMs and evolutionary computation, they typically lack parameter-level adaptation or multi-model dynamics. Our contribution is to close this gap by combining evolutionary search with experience-driven training and

collaborative co-evolution across models.

5. Conclusion

We proposed MCCE, a collaborative co-evolutionary framework that combines a frozen large language model with a trainable local model for multi-objective discrete optimization. By coupling global exploration with experience-driven local learning in a closed feedback loop, MCCE achieves performance beyond one-way distillation and consistently outperforms strong baselines in multi-objective drug design. More broadly, our results suggest that hybrid systems integrating static and adaptive models offer a promising direction for complex optimization problems, with potential extensions to other discrete domains.

Limitations

Our current validation is primarily centered on molecular optimization, and we additionally include results on MOTSP, MOCVRP, and Circle Packing as evidence of extensibility. Since the focus of this paper is large-scale discrete optimization, we will leave the exploration of continuous and mixed-variable settings, along with their associated challenges, as important directions for future work. In practice, embedding-based similarity already serves as a strong and general-purpose solution across tasks. When domain-specific similarity measures are available, they may further improve performance, and this can be explored depending on the application domain.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

Acknowledgements

This work is supported by the Zhongguancun Academy, (Grant No.s C20250203) and the Engineering and Physical Sciences Research Council (EPSRC) [EP/Y030826/1]. The EPSRC support is provided through the author’s doctoral training programme under the supervision of Richard Allmendinger and Mauricio Alvarez.

References

- Blank, J. and Deb, K. Pymoo: Multi-objective optimization in python. *Ieee access*, 8:89497–89509, 2020.
- Dat, P. V. T., Doan, L., and Binh, H. T. T. Hsevo: Elevating automatic heuristic design with diversity-driven harmony search and genetic algorithm using llms. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 26931–26938, 2025.
- Dong, Y., Jiang, X., Tao, Y., Liu, H., Zhang, K., Mou, L., Cao, R., Ma, Y., Chen, J., Li, B., et al. RL-plus: Countering capability boundary collapse of llms in reinforcement learning with hybrid-policy optimization. *arXiv preprint arXiv:2508.00222*, 2025.
- Ertl, P. and Schuffenhauer, A. Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions. *Journal of cheminformatics*, 1(1):8, 2009.
- Gao, B., Huang, Y., Liu, Y., Xie, W., Ma, W.-Y., Zhang, Y.-Q., and Lan, Y. Pushing the boundaries of structure-based drug design through collaboration with large language models. *arXiv preprint arXiv:2503.01376*, 2025a.
- Gao, H.-a., Geng, J., Hua, W., Hu, M., Juan, X., Liu, H., Liu, S., Qiu, J., Qi, X., Wu, Y., et al. A survey of self-evolving agents: On path to artificial super intelligence. *arXiv preprint arXiv:2507.21046*, 2025b.
- Huang, Y., Feng, X., Li, B., Xiang, Y., Wang, H., Liu, T., and Qin, B. Ensemble learning for heterogeneous large language models with deep parallel collaboration. *Advances in Neural Information Processing Systems*, 37: 119838–119860, 2024.
- Huang, Z., Wu, W., Wu, K., Wang, J., and Lee, W.-B. Calm: Co-evolution of algorithms and language model for automatic heuristic design. *arXiv preprint arXiv:2505.12285*, 2025.
- Jensen, J. H. A graph-based genetic algorithm and generative model/monte carlo tree search for the exploration of chemical space. *Chemical science*, 10(12):3567–3572, 2019.
- Jiang, Z., Schmidt, D., Srikanth, D., Xu, D., Kaplan, I., Jacenko, D., and Wu, Y. Aide: Ai-driven exploration in the space of code. *arXiv preprint arXiv:2502.13138*, 2025.
- Kim, H., Kim, M., Choi, S., and Park, J. Genetic-guided gflownets for sample efficient molecular optimization. *Advances in Neural Information Processing Systems*, 37: 42618–42648, 2024.
- Li, Y., Shen, X., Yao, X., Ding, X., Miao, Y., Krishnan, R., and Padman, R. Beyond single-turn: A survey on multi-turn interactions with large language models. *arXiv preprint arXiv:2504.04717*, 2025.
- Lin, X., Yang, Z., and Zhang, Q. Pareto set learning for neural multi-objective combinatorial optimization. *arXiv preprint arXiv:2203.15386*, 2022.
- Liu, F., Tong, X., Yuan, M., Lin, X., Luo, F., Wang, Z., Lu, Z., and Zhang, Q. Evolution of heuristics: Towards efficient automatic algorithm design using large language model. *arXiv preprint arXiv:2401.02051*, 2024.
- Liu, X., Xu, N., Chen, M., and Xiao, C. Autodan: Generating stealthy jailbreak prompts on aligned large language models. *arXiv preprint arXiv:2310.04451*, 2023.
- Liu, Z., Cai, Y., Zhu, X., Zheng, Y., Chen, R., Wen, Y., Wang, Y., Chen, S., et al. ML-master: Towards ai-for-ai via integration of exploration and reasoning. *arXiv preprint arXiv:2506.16499*, 2025a.
- Liu, Z., Chai, J., Zhu, X., Tang, S., Ye, R., Zhang, B., Bai, L., and Chen, S. ML-agent: Reinforcing llm agents for autonomous machine learning engineering. *arXiv preprint arXiv:2505.23723*, 2025b.
- Lu, Y., Ji, Z., Du, J., Shanqing, Y., Xuan, Q., and Zhou, T. From llm-ination to llm-orchestrator: Coordinating small models for data labeling. *arXiv preprint arXiv:2506.16393*, 2025.
- Luo, J., Zhang, W., Yuan, Y., Zhao, Y., Yang, J., Gu, Y., Wu, B., Chen, B., Qiao, Z., Long, Q., et al. Large language model agent: A survey on methodology, applications and challenges. *arXiv preprint arXiv:2503.21460*, 2025.
- Ma, L., Liang, H., Qiang, M., Tang, L., Ma, X., Wong, Z. H., Niu, J., Shen, C., He, R., Cui, B., et al. Learning what reinforcement learning can’t: Interleaved online fine-tuning for hardest questions. *arXiv preprint arXiv:2506.07527*, 2025.

- Misaki, K., Inoue, Y., Imajuku, Y., Kuroki, S., Nakamura, T., and Akiba, T. Wider or deeper? scaling llm inference-time compute with adaptive branching tree search. *arXiv preprint arXiv:2503.04412*, 2025.
- Novikov, A., Vū, N., Eisenberger, M., Dupont, E., Huang, P.-S., Wagner, A. Z., Shirobokov, S., Kozlovskii, B., Ruiz, F. J., Mehrabian, A., et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- Olivecrona, M., Blaschke, T., Engkvist, O., and Chen, H. Molecular de-novo design through deep reinforcement learning. *Journal of cheminformatics*, 9(1):48, 2017.
- Polykovskiy, D., Zhebrak, A., Sanchez-Lengeling, B., Golovanov, S., Tatanov, O., Belyaev, S., Kurbanov, R., Artamonov, A., Aladinskiy, V., Veselov, M., et al. Molecular sets (moses): a benchmarking platform for molecular generation models. *Frontiers in pharmacology*, 11:565644, 2020.
- Ran, N., Wang, Y., Zhang, X., Li, Z., Ran, Q., Li, W., and Allmendinger, R. Exllm: Experience-enhanced llm optimization for molecular design and beyond. *arXiv preprint arXiv:2502.12845*, 2025.
- Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., et al. Mathematical discoveries from program search with large language models. *Nature*, 625 (7995):468–475, 2024.
- Shin, D.-H., Son, Y.-H., Lee, D.-J., Han, J.-W., and Kam, T.-E. Dynamic many-objective molecular optimization: Unfolding complexity with objective decomposition and progressive optimization. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 6026–6034, 2024.
- Su, H., Chen, R., Tang, S., Yin, Z., Zheng, X., Li, J., Qi, B., Wu, Q., Li, H., Ouyang, W., et al. Many heads are better than one: Improved scientific idea generation by a llm-based multi-agent system. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 28201–28240, 2025.
- Sun, W., Feng, S., Li, S., and Yang, Y. Co-bench: Benchmarking language model agents in algorithm search for combinatorial optimization. *arXiv preprint arXiv:2504.04310*, 2025.
- Surina, A., Mansouri, A., Quaadvlieg, L., Seddas, A., Viazovska, M., Abbe, E., and Gulcehre, C. Algorithm discovery with llms: Evolutionary search meets reinforcement learning. *arXiv preprint arXiv:2504.05108*, 2025.
- Wang, H., Polo, F. M., Sun, Y., Kundu, S., Xing, E., and Yurochkin, M. Fusing models with complementary expertise. *arXiv preprint arXiv:2310.01542*, 2023.
- Wang, H., Skreta, M., Ser, C.-T., Gao, W., Kong, L., Strieth-Kalthoff, F., Duan, C., Zhuang, Y., Yu, Y., Zhu, Y., et al. Efficient evolutionary search over chemical space with large language models. *arXiv preprint arXiv:2406.16976*, 2024.
- Wu, J., Liao, C., Feng, M., Zhang, S., Wen, Z., Shao, P., Xu, H., and Tao, J. Thought-augmented policy optimization: Bridging external guidance and internal capabilities. *arXiv preprint arXiv:2505.15692*, 1(8):10, 2025a.
- Wu, S., Galley, M., Peng, B., Cheng, H., Li, G., Dou, Y., Cai, W., Zou, J., Leskovec, J., and Gao, J. Collabllm: From passive responders to active collaborators. *arXiv preprint arXiv:2502.00640*, 2025b.
- Wu, X., Mao, X., Li, F., Zhang, X., Zhang, X., Zhou, J., Peng, Y., Zheng, L., Teng, C., Ji, D., et al. Evorefuse: Evolutionary prompt optimization for evaluation and mitigation of llm over-refusal to pseudo-malicious instructions. *arXiv preprint arXiv:2505.23473*, 2025c.
- Xu, R., Shi, W., Zhuang, Y., Yu, Y., Ho, J. C., Wang, H., and Yang, C. Collab-rag: Boosting retrieval-augmented generation for complex question answering via white-box and black-box llm collaboration. *arXiv preprint arXiv:2504.04915*, 2025.
- Yan, J., Li, Y., Hu, Z., Wang, Z., Cui, G., Qu, X., Cheng, Y., and Zhang, Y. Learning to reason under off-policy guidance. *arXiv preprint arXiv:2504.14945*, 2025.
- Yang, S., Li, Y., Lam, W., and Cheng, Y. Multi-llm collaborative search for complex problem solving. *arXiv preprint arXiv:2502.18873*, 2025.
- Yao, S., Liu, F., Lin, X., Lu, Z., Wang, Z., and Zhang, Q. Multi-objective evolution of heuristic using large language model. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 27144–27152, 2025.
- Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., and Song, G. Reevo: Large language models as hyper-heuristics with reflective evolution. *Advances in neural information processing systems*, 37:43571–43608, 2024.
- Ye, P., Jiang, Y., Yang, S., Tang, S., Zheng, S., Cao, J., Ren, S., Dou, Z., Wang, W., He, H., Li, H., Cui, Z., Zhong, Z., Wu, M., Zhang, S., Zhou, D., Zhang, B., Hu, S., Chen, T., and Bai, L. A comprehensive survey of llm-driven collective intelligence: Past, present, and future, 08 2025.

Zhao, J., Wen, T., and Hao Cheong, K. Can large language models be trusted as black-box evolutionary optimizers for combinatorial problems? *arXiv e-prints*, pp. arXiv–2501, 2025.

Zheng, J., Shi, C., Cai, X., Li, Q., Zhang, D., Li, C., Yu, D., and Ma, Q. Lifelong learning of large language model based agents: A roadmap. *arXiv preprint arXiv:2501.07278*, 2025.

A. Appendix

A.1. Prompt Example

Suggest new molecules that satisfy the following requirements: 1. decrease the SA value. 2. decrease the DRD2 value. 3. increase the QED value. 4. decrease the GSK3 β value. 5. increase the JNK3 value.

sa: SA measures how easily a molecule can be synthesized based on its structural complexity. Simplifying a molecule by reducing complex ring systems or functional groups can lower SA, making synthesis easier, while adding complex structures can increase SA, making synthesis harder.

drd2: Dopamine receptor D2 (DRD2) is a receptor involved in the modulation of neurotransmission and is a target for various psychiatric and neurological disorders. Adding functional groups like hydroxyl or halogen atoms to aromatic rings can enhance binding affinity to DRD2. Removing aromaticity or introducing bulky groups near the binding sites often decreases DRD2 activity.

qed: QED (Quantitative Estimate of Drug-likeness) is a measure that quantifies how 'drug-like' a molecule is based on properties such as molecular weight, solubility, and the number of hydrogen bond donors and acceptors. Adding functional groups that improve drug-like properties (e.g., small molecular size, balanced hydrophilicity) can increase QED, while introducing large, complex, or highly polar groups can decrease it.

gsk3b: Glycogen synthase kinase-3 beta (GSK3 β) is an enzyme involved in cellular processes like metabolism and apoptosis, and is a therapeutic target for cancer and neurological diseases. Adding polar groups, such as hydroxyls, can improve hydrogen bonding with GSK3 β 's active site. Introducing steric hindrance or highly hydrophobic regions can reduce interactions with GSK3 β .

jnk3: c-Jun N-terminal kinase 3 (JNK3) is a kinase involved in stress signaling and is targeted for neuroprotection in diseases like Alzheimer's. Introducing small polar or electronegative groups can enhance binding affinity to JNK3. Removing polar functional groups or adding large, bulky substituents can reduce activity by obstructing the active site.

Give me 2 new molecules that fit the features.

You can do it by applying crossover on the given points and based on your knowledge. The molecule should be valid.

Do not write code. Do not give any explanation. Each output new molecule must start with mol and end with /mol in SIMLES form. Your answer can only contain two molecules and end immediately.

A.2. DPO loss Analysis

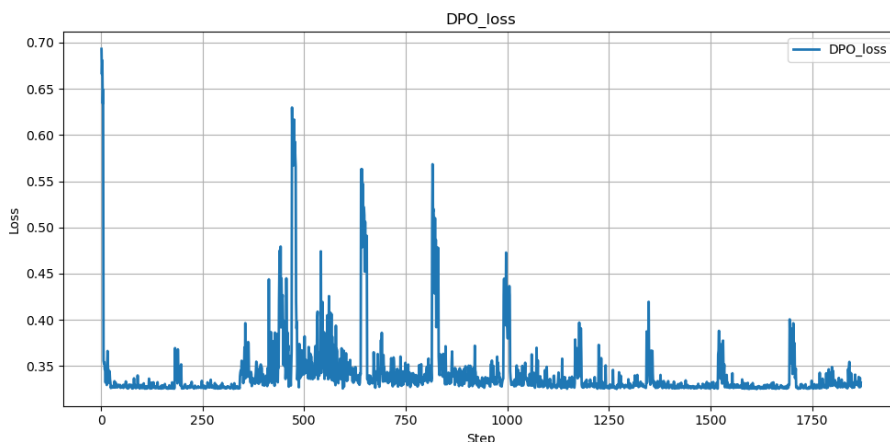


Figure 4. loss Analysis

Figure 4 plots the training loss curve of our DPO optimization. We observe that as the training step increases, the overall loss gradually decreases and the peak values become progressively lower. This trend indicates that the local model is steadily learning and aligning with the distribution of high-quality molecules. The occasional sharp peaks correspond to the

introduction of newly synthesized training data, which temporarily increases the difficulty of optimization. Importantly, the diminishing magnitude of these peaks over time reflects that the model is effectively absorbing new knowledge while maintaining stability, thereby confirming the robustness of our similarity-based data synthesis strategy.

A.3. Training Details

To ensure stability during Direct Preference Optimization (DPO) training, we adopt the *initial untrained local model* π_{θ_0} as the reference model π_{ref} in the DPO loss. For each training triplet (q, τ^+, τ^-) , the DPO objective is

$$\ell_{\text{DPO}}(q, \tau^+, \tau^-) = -\log \sigma \left(\beta \left[\log \frac{\pi_{\theta}(\tau^+ | q)}{\pi_{\theta_0}(\tau^+ | q)} - \log \frac{\pi_{\theta}(\tau^- | q)}{\pi_{\theta_0}(\tau^- | q)} \right] \right), \quad (11)$$

where $\sigma(\cdot)$ is the sigmoid function and $\beta > 0$ is a scaling parameter. By fixing $\pi_{\text{ref}} = \pi_{\theta_0}$, we prevent drift of the reference distribution and guarantee that the optimization process always measures progress relative to the original model. This prevents instability that might occur if the reference model itself were updated during training. In practice, we observe a monotonically decreasing average loss curve, which provides evidence that the local model is gradually aligning with the distribution of high-quality molecules.

Training frequency and dataset size. We denote by f the update frequency (number of generated candidates between two training updates) and by $|\mathcal{D}|$ the size of the synthesized DPO dataset. Both hyperparameters significantly influence stability and performance. Empirically, smaller f (i.e., more frequent updates) accelerates adaptation but may introduce variance due to limited data per update, while larger $|\mathcal{D}|$ provides smoother gradients at the cost of slower responsiveness.

Comparison across paradigms. We conducted extensive hyperparameter sweeps for several training paradigms, including SFT, offline RL, GFlowNets, and our DPO method. Let $\mathcal{M}$ denote the set of all hyperparameter configurations explored for a given method m . The optimal performance is reported as

$$\text{Perf}(m) = \max_{\lambda \in \mathcal{M}} \mathbb{E}[s(c) | c \sim \pi_{m,\lambda}], \quad (12)$$

where $s(c)$ is the evaluation score of molecule c and $\pi_{m,\lambda}$ is the trained model with hyperparameter configuration λ . Across all settings, our DPO-based approach consistently achieved higher $\text{Perf}(m)$ than SFT and offline RL, and demonstrated greater robustness to hyperparameter variations.

A.4. Detailed Algorithm for Similarity-Based Data Synthesis

For completeness, we provide the full pseudocode of the per-prompt DPO pair construction procedure, including all fallback rules and implementation details.

A.5. Additional Co-evolutionary Curves

To further validate the effectiveness of our collaborative co-evolution framework, we provide four additional co-evolutionary curves in Figure 5. These curves consistently demonstrate the same trend observed in the main text: the frozen large LLM provides broad exploration by leveraging its prior knowledge, while the local model—progressively refined through experience learning—contributes focused exploitation and adaptation to the evolving search space. The alternating interplay between exploration and exploitation prevents premature convergence, enhances diversity, and steadily drives the optimization toward superior solutions. Importantly, across all cases, the collaborative trajectory consistently surpasses that of either model operating alone, confirming the robustness and generality of the co-evolutionary mechanism.

A.6. Supplementary Analysis of Synthetic Accessibility

To assess the practical applicability of the molecules generated by our MCCE framework, we rigorously evaluated their Synthetic Accessibility (SA). We utilized the SA Oracle provided by the *Therapeutics Data Commons* (TDC)¹, instantiated via `Oracle(name = 'SA')`.

The SA score estimates the difficulty of synthesizing a given molecule based on a combination of the molecule’s fragment contributions and molecular complexity. The metric is calculated via RDKit using a set of chemical rules originally defined

¹<https://tdcommons.ai/functions/oracles/#synthetic-accessibility-sa>

Algorithm 2 Per-Prompt DPO Pair Construction with Fallback Rules

```

1: Input: Historical prompts  $\mathcal{H}$ , candidate sets  $\{\mathcal{C}_q\}$ , similarity filter  $\mathcal{F}$ , high/low score pools  $\mathcal{T}_{\text{high}}, \mathcal{T}_{\text{low}}$ , similarity
   intervals  $I_1, I_2, I_3$ , max recent prompts  $L$ , max pairs per prompt  $r$ 
2: Output: Set of DPO triplets  $\{(q, \tau^+, \tau^-)\}$ 
3: Select the most recent  $L$  prompts from  $\mathcal{H}$ 
4: for each prompt  $q$  in selected prompts do
5:    $\mathcal{C}_q^{\mathcal{F}} \leftarrow \mathcal{C}_q \cap \mathcal{F}$ 
6:   for  $i = 1$  to  $r$  do
7:     // Select preferred trajectory  $\tau^+$ 
8:     Try selecting  $\tau^+$  from  $\mathcal{C}_q^{\mathcal{F}} \cap \mathcal{T}_{\text{high}} \cap I_1$ 
9:     if  $\tau^+$  not found then
10:      Relax similarity to  $I_2$ 
11:      if still not found then
12:        Relax similarity to  $I_3$ 
13:        if still not found then
14:          Broaden to Top-50% score pool
15:        end if
16:      end if
17:    end if
18:    If multiple candidates satisfy, choose the highest-scoring or sample uniformly
19:    // Select rejected trajectory  $\tau^-$ 
20:    Try selecting  $\tau^-$  from  $\mathcal{C}_q^{\mathcal{F}} \cap \mathcal{T}_{\text{low}} \cap I_1$ 
21:    if  $\tau^-$  not found then
22:      Relax similarity to  $I_2$ , then  $I_3$ , then Bottom-50% score pool
23:    end if
24:    // Validation
25:    if  $\tau^+$  or  $\tau^-$  missing then
26:      Optionally skip this prompt or sample randomly from the corresponding 50% pool
27:    end if
28:    Record triplet  $(q, \tau^+, \tau^-)$  and optionally store  $s(\tau^\pm)$  and  $\text{sim}(\tau^\pm, q)$ 
29:  end for
30: end for

```

by Ertl & Schuffenhauer (2009). This implementation is widely adopted in molecular generation benchmarks, including the Molecular Sets (MOSES) platform (Polykovskiy et al., 2020).

The scoring scale follows these empirical guidelines:

- **1.0 – 3.0:** Easy to synthesize.
- **3.0 – 6.0:** Intermediate to difficult.
- **> 6.0:** Very difficult or impossible to synthesize.

A lower score indicates better synthesizability, which is a critical constraint in real-world drug discovery pipelines.

We conducted a statistical analysis on the final populations (‘final_pops’) obtained after 5 independent evolutionary runs. The aggregated statistics for the Synthetic Accessibility scores are presented in Table 7.

Table 7. Statistical Analysis of Synthetic Accessibility (SA) Scores in Final Populations.

Metric	Count (N)	Mean	Std. Dev.	Min	Max
Synthetic Accessibility	250	2.00	0.22	1.49	3.06

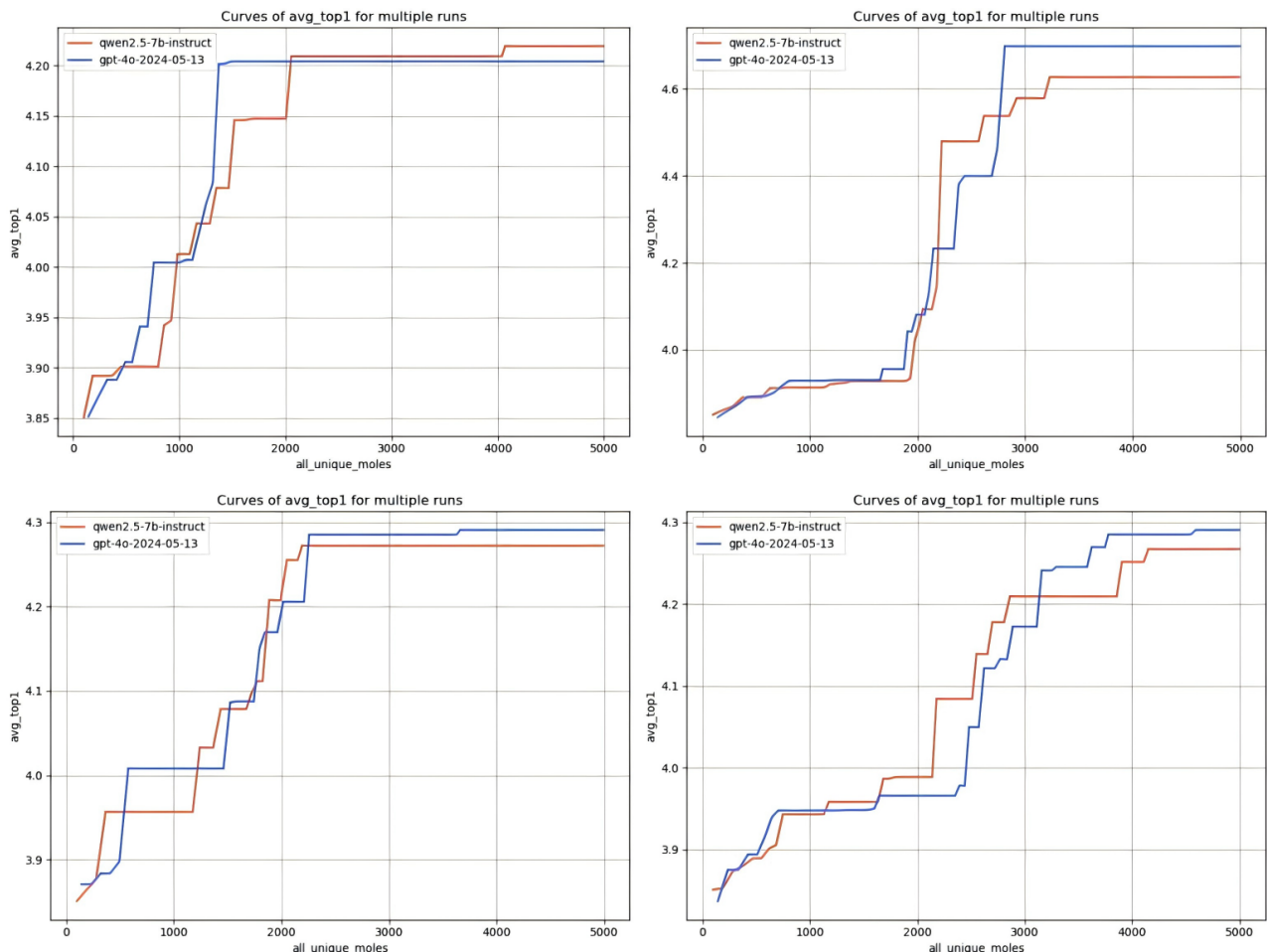


Figure 5. Additional Co-evolutionary Curves

As shown in Table 7, the generated molecules exhibit exceptional synthesizability profiles:

High Synthesizability: The mean SA score is **2.00**, which falls well within the "easy to synthesize" range (1–3). This is significantly lower than the typical threshold for intermediate difficulty, suggesting that the generated candidates are chemically realistic and practical for wet-lab synthesis.

Concentrated Distribution: The standard deviation is low (**0.22**), and the range is narrow ([1.49, 3.06]). This indicates that the MCCE framework maintains a tight control over the chemical complexity of the population. Unlike traditional evolutionary methods that might exploit scoring functions by generating overly complex or chaotic structures, our collaborative co-evolution approach effectively filters out "hard-to-synthesize" outliers.

Successful Multi-Objective Constraint: It is important to note that these favorable SA scores were achieved while simultaneously optimizing four other challenging objectives (DRD2, QED, GSK3 β , and JNK3). The fact that the maximum SA score observed is only **3.06** (borderline intermediate) demonstrates that MCCE successfully treats SA as a hard constraint. The local trainable model, refined via DPO, appears to have internalized the implicit rules of chemical validity and simplicity, avoiding the generation of unrealistic high-scoring artifacts.

In conclusion, the SA analysis confirms that the MCCE framework produces high-quality molecular candidates that are not only theoretically potent (high binding affinity) but also practically viable for synthesis.

A.7. Cost Analysis

To demonstrate the cost-effectiveness of the MCCE framework, we tracked the detailed computational resources and financial costs associated with a standard evolutionary run. The experiment was conducted using the **GPT-4o-2024-05-13** model as the frozen global explorer and a local trainable model **Qwen2.5-7B-Instruct** on a high-performance compute node.

The breakdown of the computational budget and incurred costs for generating a total of 5,000 candidates (‘Budget_candidates’) is summarized in Table 8.

Table 8. Computational Cost and Resource Usage for MCCE (GPT-4o + Local Model). Statistics are reported as Mean $\pm$ Std over multiple runs.

Metric	Value / Specification
Target Population Budget	5,000 Candidates
Model Configuration (API / Local)	50 / 32
Hardware Infrastructure	8 $\times$ NVIDIA A800 (40GB)
Total LLM Calls	3471.14 $\pm$ 165.34
Total Wall-clock Time (Hours)	3.12 $\pm$ 0.31
Total API Cost (USD)	\$3.814 $\pm$ 0.457

The data in Table 8 highlights several key advantages of the collaborative co-evolution paradigm:

High Cost-Efficiency: The total financial cost for accessing the proprietary closed-source model (GPT-4o) was remarkably low, averaging approximately **\$3.81 per run**. This is significantly more economical than pure API-based evolutionary methods, which typically require extensive token consumption for every generation step. By offloading a significant portion of the localized search and exploitation to the local trainable model, MCCE drastically reduces the dependency on expensive API calls.

Reasonable Time Complexity: Utilizing an 8 $\times$ A800 GPU cluster, the entire evolutionary process (including DPO fine-tuning and candidate evaluation) concluded in roughly **3.12 hours**. This demonstrates that the framework is computationally feasible for iterative scientific discovery loops, where rapid turnaround is essential.

Effective Collaboration: The total number of LLM calls ($\approx 3,471$) relative to the candidate budget (5,000) suggests an efficient generation strategy. The difference implies that the system effectively utilizes crossover, mutation, and the local model’s generation capabilities to expand the population, further optimizing the resource-to-performance ratio.

These findings confirm that MCCE offers a scalable and sustainable path for leveraging Large Language Models in optimization, minimizing the “token tax” usually associated with state-of-the-art foundation models.

A.8. Impact of Similarity Metrics and DPO Stability Analysis

A core finding of our experiments is that the stability of Direct Preference Optimization (DPO) within a co-evolutionary framework is heavily dependent on the structural similarity of the training pairs. Regardless of the specific mathematical definition of the metric, as long as the metric effectively reflects the pairwise similarity between the chosen (τ^+) and rejected (τ^-) trajectories, it significantly stabilizes the training process.

Our experiments demonstrate that DPO training data synthesis without similarity constraints leads to high variance and gradient conflicts, as the model is forced to compare candidates from disjoint distributions. In contrast, enforcing a similarity constraint ensures that the model learns from consistent, local improvements (i.e., “how to make a good molecule slightly better”) rather than confusing global jumps.

Figure 6 visualizes this phenomenon. The loss curve without similarity filtering (a) exhibits severe oscillations and instability, whereas the curve with similarity-based data synthesis (b) remains smooth and converges steadily.

While our primary experiments on molecular optimization utilized domain-specific molecular fingerprints (e.g., Tanimoto similarity on Morgan fingerprints), we extended our framework to other combinatorial and geometric domains by adopting **embedding-based similarity metrics**.

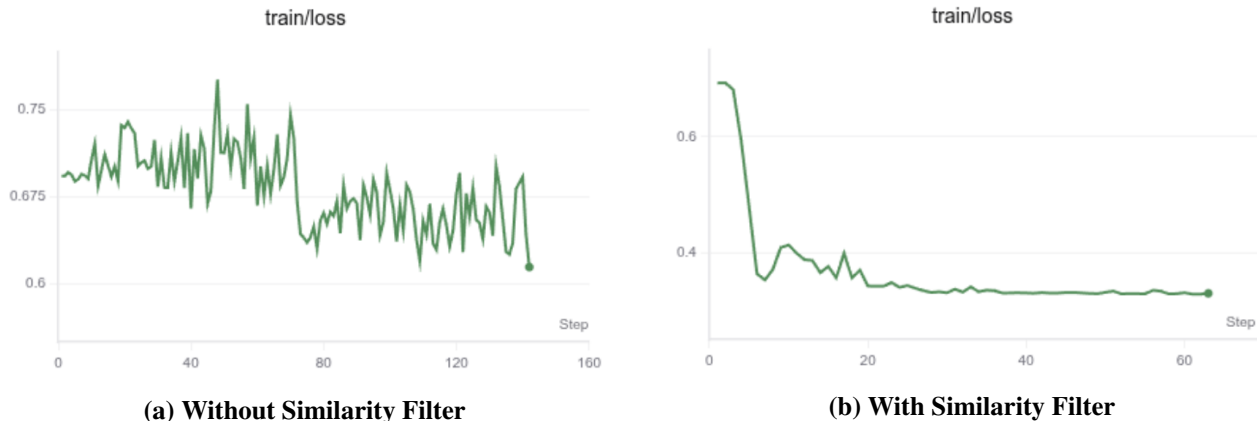


Figure 6. Comparison of DPO training stability. (a) Without similarity constraints, the loss is highly volatile due to distributional mismatch. (b) With our similarity-based data synthesis, the training loss is stable and converges smoothly, confirming the effectiveness of the proposed strategy.

In these supplementary tasks, we found that mapping the decision variables (e.g., routes, coordinates) into a latent embedding space and computing Cosine Similarity yielded equally effective results. This confirms that the MCCE framework is not tied to a specific chemical metric but is a generalizable paradigm dependent only on a reliable measure of "distance" in the solution space. Table 9 details the specific configurations used for each task type.

Table 9. Similarity metrics and configurations used across different optimization tasks. While molecular tasks rely on discrete fingerprints, continuous and combinatorial tasks utilize embedding-based cosine similarity.

Task Type	Similarity Metric	Embedding Content	Normalization	Similarity Range
Circle Packing	Cosine Similarity	Circle centers + radii	L2 normalization	[0, 1]
Molecule Optimization	TDC Similarity_Meta	Molecular fingerprints (SMILES)	N/A (external library)	[0, 1]
Vehicle Routing	Cosine Similarity	Per-route (customer count, demand, distance)	L2 normalization	[0, 1]
Traveling Salesman	Cosine Similarity	Edge length sequences under two objectives	L2 normalization	[0, 1]

A.9. Details of Multi-Objective Selection Mechanism

To strictly balance convergence quality and population diversity—thereby maximizing the Hypervolume (HV) indicator—we designed a **Hybrid Elite-Diversity Selection Strategy**. This strategy constructs the next-generation population of size N by combining Single-Objective (SO) optimization with Pareto-based diversity maintenance.

The population construction is divided into two phases:

- **Elite Preservation (Top 50%):** To ensure rapid convergence towards high-fitness regions, the first half of the population ($N/2$) is selected solely based on the aggregated total score (SO Selection). This acts as a strong exploitation signal.
- **Diversity Maintenance (Bottom 50%):** To prevent the population from collapsing into a single mode and to cover the Pareto front widely, the remaining $N/2$ slots are filled using candidates from the optimal Pareto layers. In this phase, we enforce strict duplicate removal to guarantee structural uniqueness.

The detailed algorithmic flow is as follows:

1. **Elite Selection:** Sort all candidates in the current pool by their aggregated fitness score $S(c)$. Select the top $N/2$ individuals to form the elite set $\mathcal{P}_{elite}$.
2. **Pareto Layering:** Perform Non-Dominated Sorting on the entire candidate pool to partition it into Pareto ranks $\mathcal{R}_1, \mathcal{R}_2, \dots, \mathcal{R}_k$, where $\mathcal{R}_1$ represents the non-dominated front.

3. **Diversity Filling:** Initialize the diversity set $\mathcal{P}_{div} = \emptyset$. Iterate through ranks $i = 1, 2, \dots$:
 - Sort candidates within $\mathcal{R}_i$ by total score.
 - Sequentially add candidate $c \in \mathcal{R}_i$ to $\mathcal{P}_{div}$ if and only if c is chemically unique (i.e., its canonical SMILES string is not already present in $\mathcal{P}_{elite} \cup \mathcal{P}_{div}$).
 - Stop once $|\mathcal{P}_{elite}| + |\mathcal{P}_{div}| = N$.
4. **Final Population:** $\mathcal{P}_{next} = \mathcal{P}_{elite} \cup \mathcal{P}_{div}$.

By prioritizing rank-1 Pareto solutions while explicitly filtering duplicates, this method effectively maintains a diverse set of high-quality trade-off solutions, directly contributing to the superior HV and Uniqueness metrics observed in our experiments.